

Computational Physics

Lecture 2: Graphics and Visualization, Machine Precision

- Graphics and visualization (using matplotlib library)
- Accuracy of integer and floating-point number representation
- Program speed

Graphics and Visualization

- Line plots
- Scatter plots
- Density/Contour plots (2D data)
- Histogram

References:

Computational Physics by Mark Newman, Chapter 3

<https://websites.umich.edu/~mejn/cp2/chapters/chapter3.pdf>

Matplotlib documentation

<https://matplotlib.org/stable/index.html>

Plotting the data

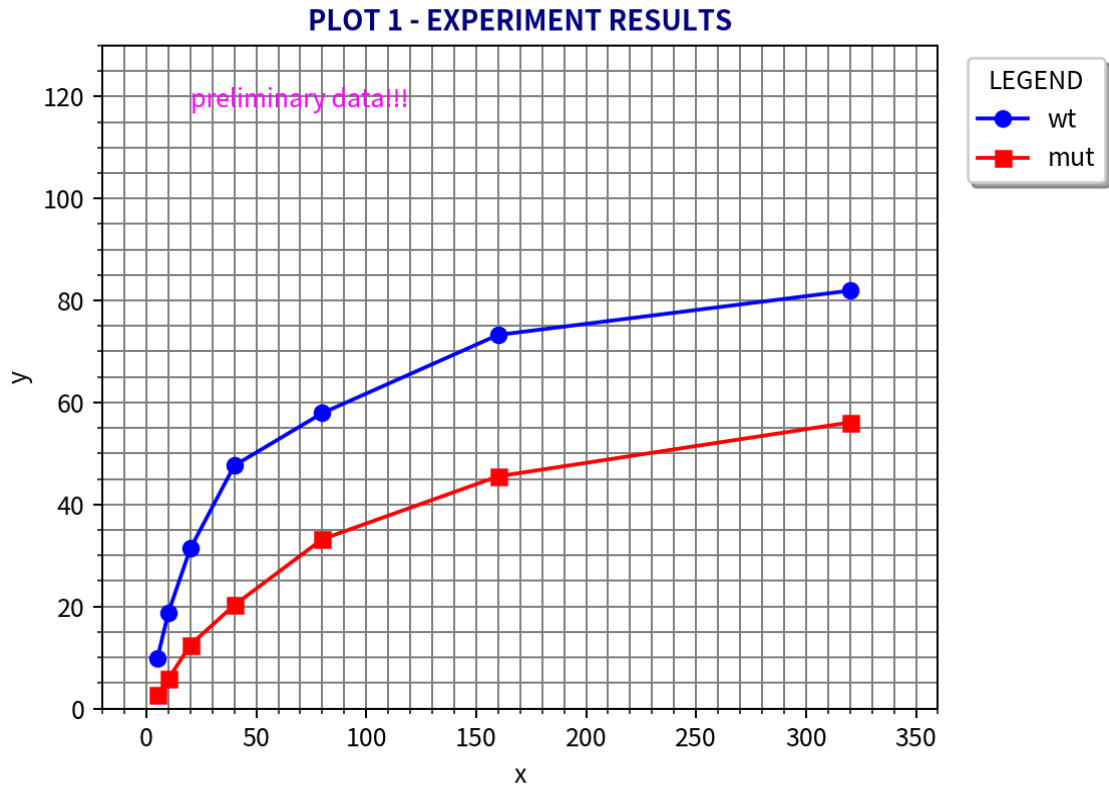
When implementing some physical system in a program, we typically produce tons of data. Looking at numbers alone do not typically make it easy to understand the behavior of the system and its properties.

In fact, it is a kind of an art to be able to understand and interpret these data. The first step in interpreting them is to **produce graphs**, i.e. a depiction of numerical data on calibrated axes.

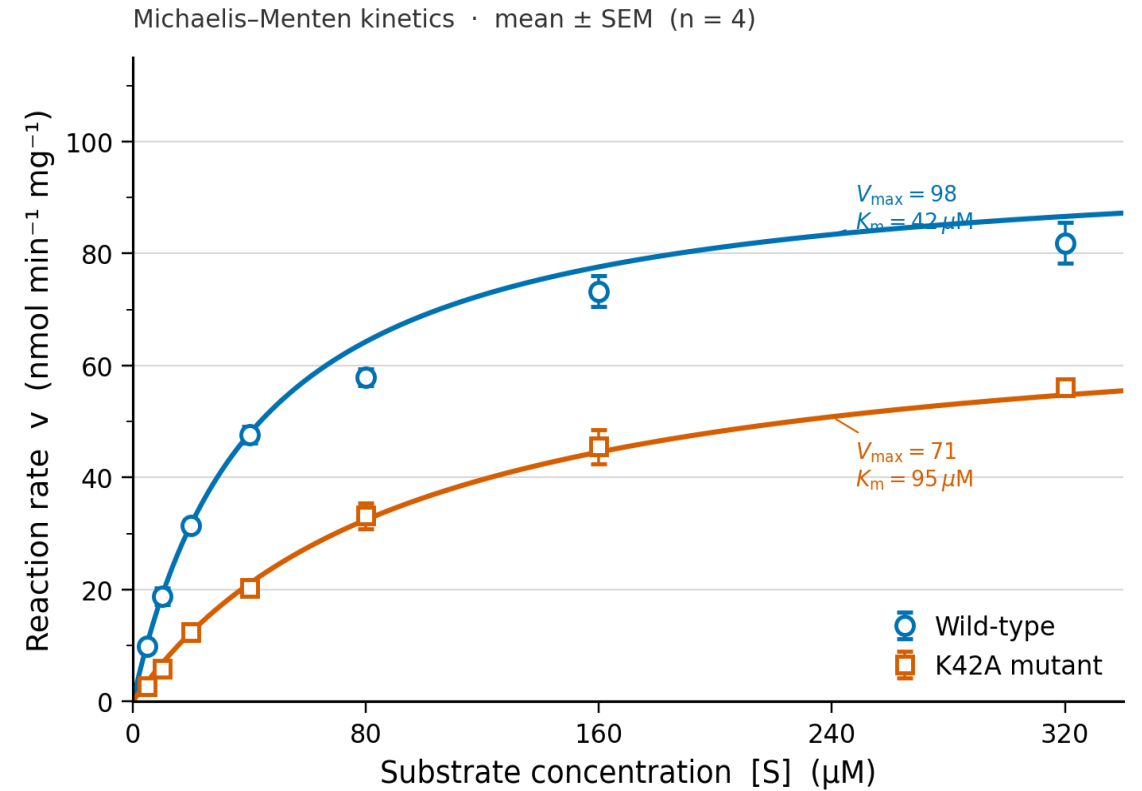
How one is able to correctly plot these data largely determine their interpretation and their readability by other scientists, especially when they need to be added in publications.

Plotting the data

BEFORE



AFTER



Plotting the data

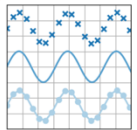
There are several Python packages for making graphs

Here we will use the popular package **matplotlib**

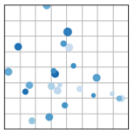
Look at https://matplotlib.org/stable/plot_types/index.html to have an idea of the possible graphs:

Pairwise data

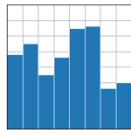
Plots of pairwise (x, y) , tabular $(var_0, \dots, var_n)$, and functional $f(x) = y$ data.



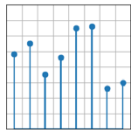
plot(x, y)



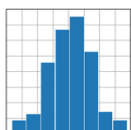
scatter(x, y)



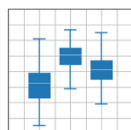
bar(x, height)



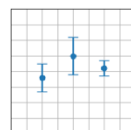
stem(x, y)



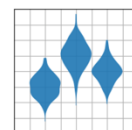
hist(x)



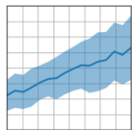
boxplot(X)



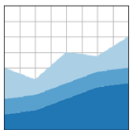
errorbar(x, y, yerr, xerr)



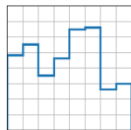
violinplot(D)



fill_between(x, y1, y2)



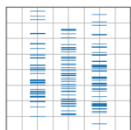
stackplot(x, y)



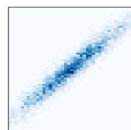
stairs(values)

Statistical distributions

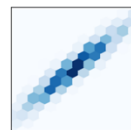
Plots of the distribution of at least one variable in a dataset. Some of these methods also compute the distributions.



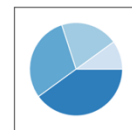
eventplot(D)



hist2d(x, y)



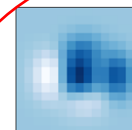
hexbin(x, y, C)



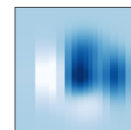
pie(x)

Gridded data

Plots of arrays and images $Z_{i,j}$ and fields $U_{i,j}$, $V_{i,j}$ on [regular grids](#) and corresponding coordinate grids $X_{i,j}$, $Y_{i,j}$.



imshow(Z)



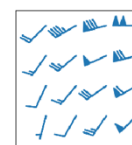
pcolormesh(X, Y, Z)



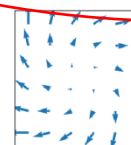
contour(X, Y, Z)



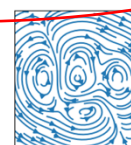
contourf(X, Y, Z)



barbs(X, Y, U, V)



quiver(X, Y, U, V)



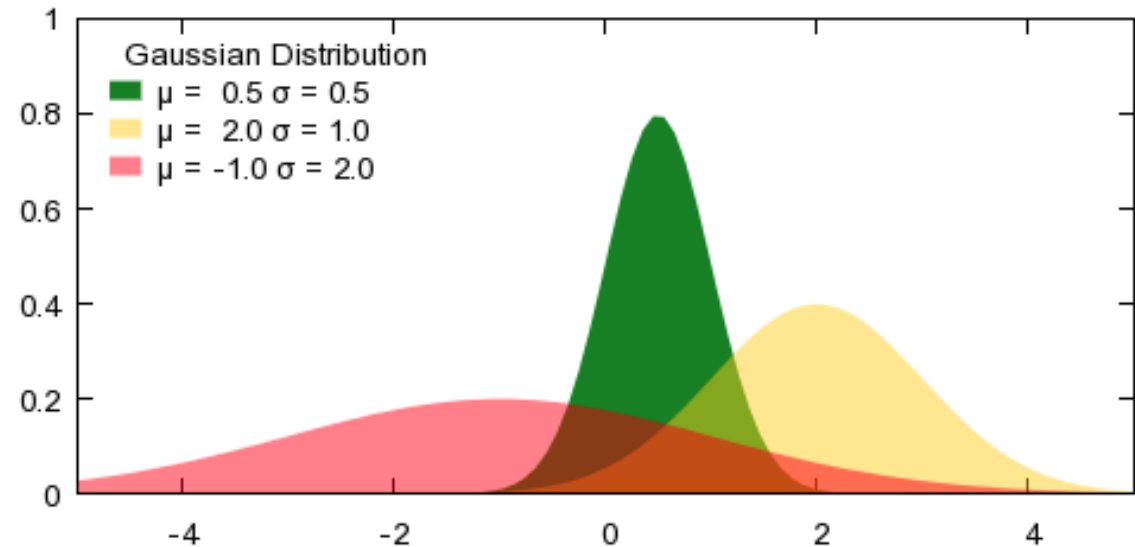
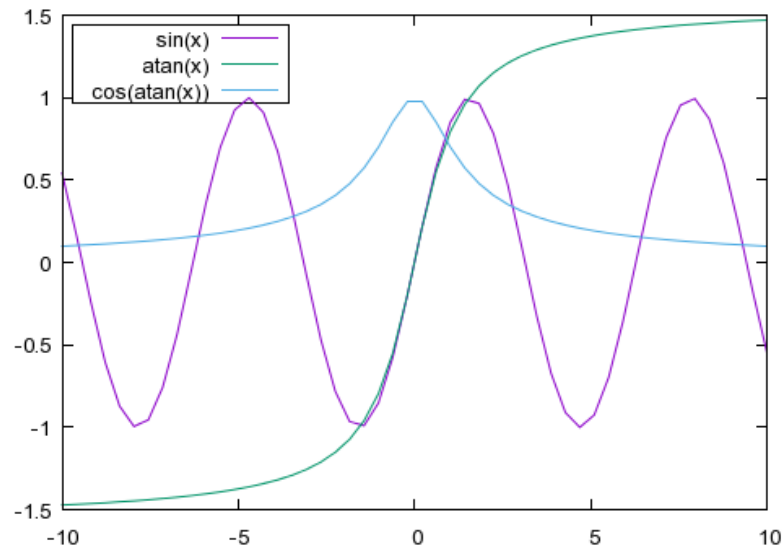
streamplot(X, Y, U, V)

Plotting the data

There are of course other solutions for plotting data. E.g. I use **gnuplot**

It has its own coding language, and you can still produce high quality graphs. And it is **interactive**

<https://gnuplot.sourceforge.net/demo/>



Line plots

Start with importing the submodule called **pyplot**:

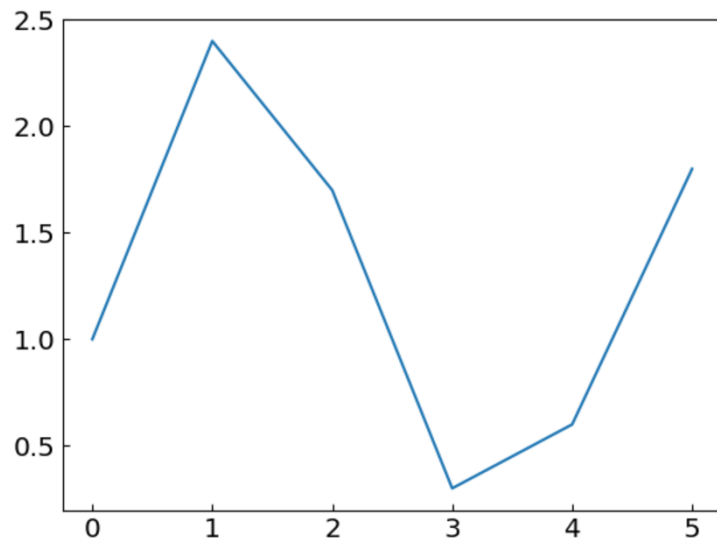
```
import matplotlib.pyplot as plt
```

To create an ordinary line plot, use the function **plt.plot**

In the simplest case, you can give the function a list or an array of the values to plot

```
y = [1.0, 2.4, 1.7, 0.3, 0.6, 1.8]    #initialize list  
plt.plot(y)    #prduce graph and store in memory (doesn't display it)  
plt.show()    #show graph
```

If you run the function, you obtain this graph displayed on a window



We have our array numbers on the y-axis, the array index on the x-axis, with points joined by lines by default

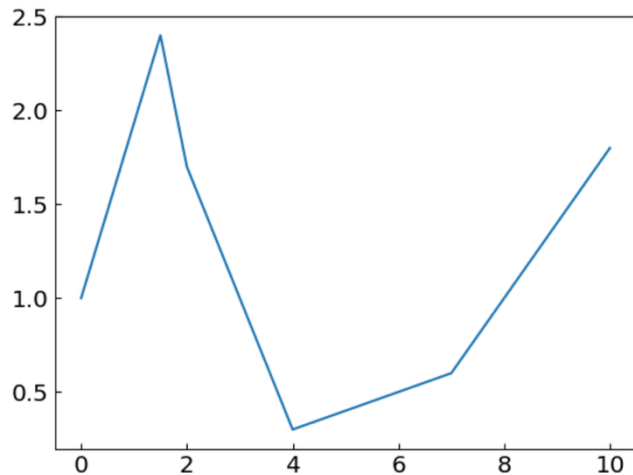
If you are running from the python console, the `plt.show()` function block the execution of the program up to that point. When the plt window is closed, the program will continue running.

Line plots

Now we specify both the x-axis and y-axis values of our points (most typical situation)

```
x = [0.0, 1.5, 2.0, 4.0, 7.0, 10.0]
y = [1.0, 2.4, 1.7, 0.3, 0.6, 1.8]
plt.plot(x,y)
plt.show()
```

which produces



Note that the two input lists need to have the same number of entries

Also, the separation between `plt.plot` and `plt.show` allow to plot two of more curves together, just by running `plt.plot` multiple times before using `plt.show`

Line plots

Let's use a more specific function: $y = \sin(x)$
and calculate it for 10 equidistant points in the interval $x = 0 \dots 10$

```
from numpy import linspace, sin, cos
```

 ← function are contained in the **numpy** library

```
# Take 10 equally spaced points between 0 and 10 and store them as a list
```

```
numpoints = 10
```

```
x = linspace(0, 10, numpoints)
```

```
# Calculate the value of sin(x) and each point and store in a list y
```

```
y = sin(x)
```

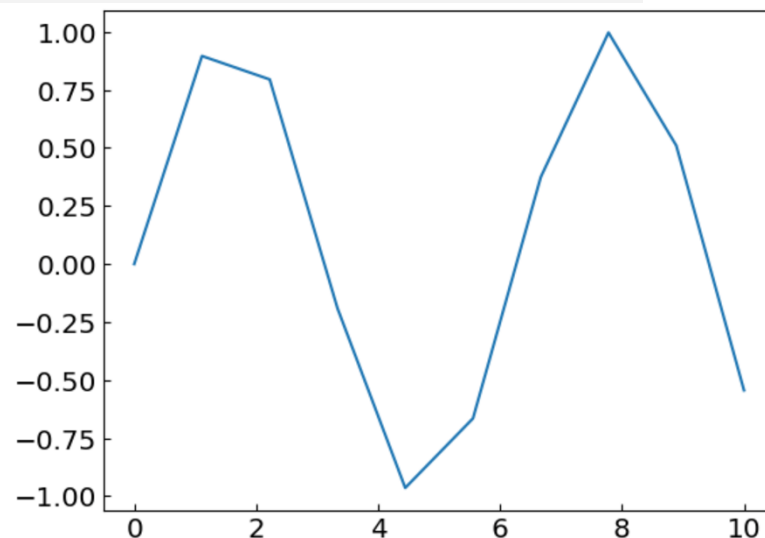
```
# Make a plot and show the results
```

```
plt.plot(x, y)
```

```
plt.show()
```

Resolution (number of points) is too low:
increase numpoints to 100

x	sin(x)
0	0.
1	0.841471
2	0.9092974
3	0.14112
4	-0.7568025
5	-0.9589243
6	-0.2794155
7	0.6569866
8	0.9893582
9	0.4121185
10	-0.5440211

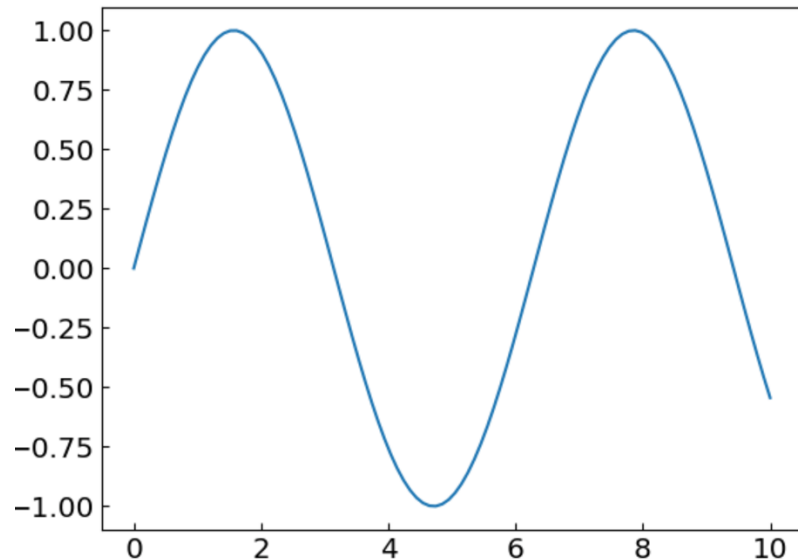


Line plots

```
numpoints = 100
x = linspace(0,10,numpoints)

# Calculate the value of sin(x) and each point and store in a list y
y = sin(x)

# Make a plot and show the results
plt.plot(x,y)
plt.show()
```



x	sin(x)
0.	0.
0.1	0.09983342
0.2	0.1986693
0.3	0.2955202
0.4	0.3894183
0.5	0.4794255
0.6	0.5646425
0.7	0.6442177
0.8	0.7173561
0.9	0.7833269
1.	0.841471
	⋮
9.9	-0.4575359
10.	-0.5440211

Now it looks like a smooth curve, but remember we are still drawing a set of points connected by lines, which becomes indistinguishable from a smooth curve with a sufficient number of points.

Line plots

If the program is small, you can typically generate your x-y data on the fly during the computation, saving them e.g. on a list and plotting the result at the end of the program.

```
#generate x-y points on the fly  
  
xpoints = []  
ypoints = []  
for x in linspace(0,10,100):  
    xpoints.append(x)  
    ypoints.append(sin(x))  
plt.plot(xpoints,ypoints)  
plt.show()
```

When the program requires however lot of computational resources and lot of waiting time, it becomes best to save the data on files, and later plot them using separate scripts

Line plots

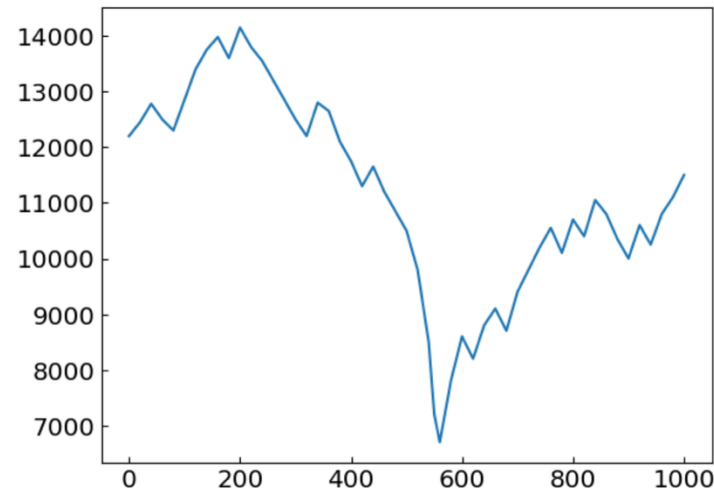
Suppose we want to **plot from a file**. We need first to load it into python as an array, and then plot the arrays.

A best practice is to have your xy file organized in the following way:

0	12121.71
1	12136.44
2	12226.73
3	12221.93
4	12194.13
5	12283.85
6	12331.60
7	12309.25
...	

We can load into a numpy array using loadtxt

```
#print x-y data importing them from a file  
  
from numpy import loadtxt  
  
data = loadtxt("values.txt",float)  
x = data[:,0]  
y = data[:,1]  
plt.plot(x,y)  
plt.show()
```



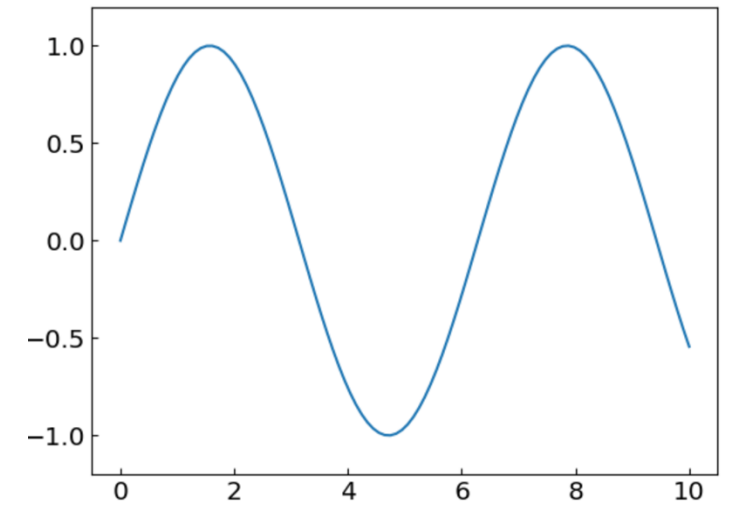
Line plots

Embellishing the plot and improving readability (for scientific publication)

1. tweak the x and y ranges of the plot. Python makes an automatic choice, which is typically good, but sometimes to see features you need to adjust the range.

Override automatic choice with **plt.xlim** and **plt.ylim**. Use it after `plt.plot` and before `plt.show`

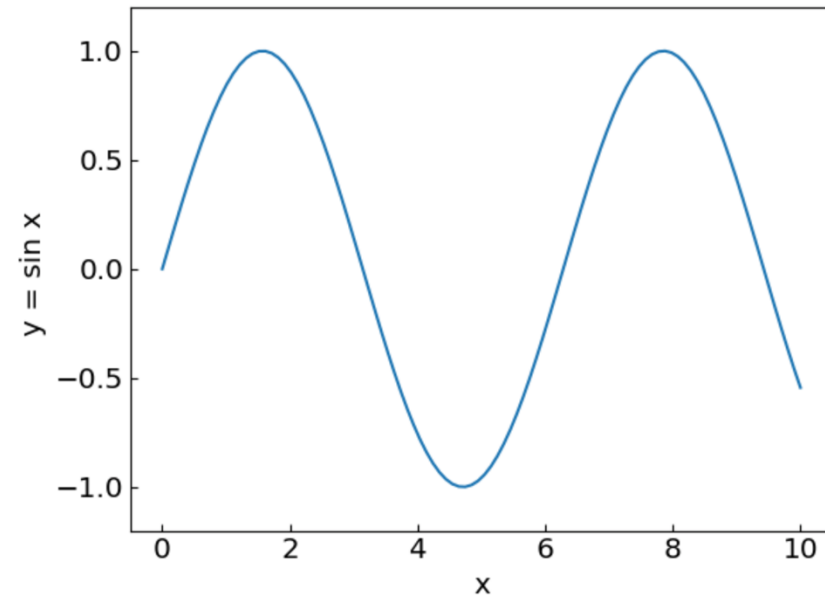
```
# Add some white space on the border redefining y-range  
x = linspace(0,10,100)  
y = sin(x)  
plt.plot(x,y)  
plt.ylim(-1.2,1.2)  
plt.show()
```



Line plots

2. Always add the **labels** indicating the quantity you are plotting. Use `plt.xlabel` and `plt.ylabel`. Otherwise, people will not understand what quantities you are talking about

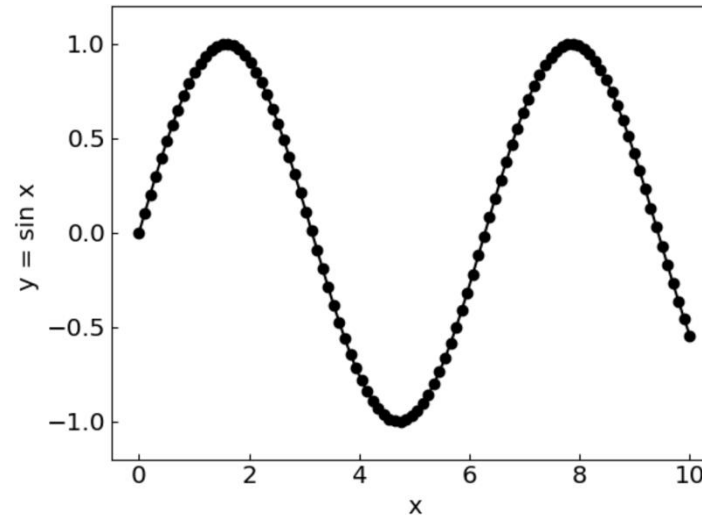
```
# Add labels
x = linspace(0,10,100)
y = sin(x)
plt.plot(x,y)
plt.ylim(-1.2,1.2)
plt.xlabel("x")
plt.ylabel("y = sin x ")
plt.show()
```



Line plots

3. Play with the line style, width, color, or use line-points. These are options to be added to `plt.plot`. e.g., add option “ko-”

```
# Change linestyle
x = linspace(0,10,100)
y = sin(x)
plt.plot(x,y,"ko-")
plt.ylim(-1.2,1.2)
plt.xlabel("x")
plt.ylabel("y = sin x ")
plt.show()
```



First letter: color of curve. r, g, b, c, m, y, k, w, are for red, green, blue, cyan, magenta, yellow, black, and white.

Second letter: symbol or marker to be placed at each data point along the curve. E.g. s, o, x, *, . for squares, circles, crosses, stars, and dots, respectively.

Third part: style of the line. “-” for a solid line, “--” for a dashed line, and “:” for a dotted line. Any part of the string can be omitted, leaving the default.

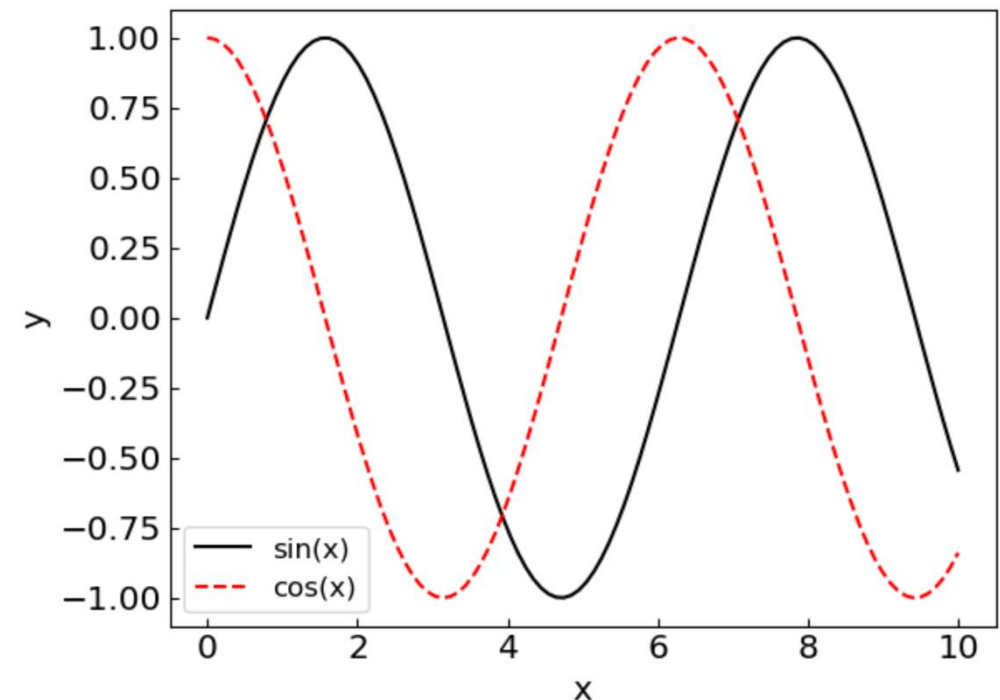
Line plots

- Plotting multiple curves together, e.g. for reference or comparison. Just add `plt.plot` for each curve. Change the style to better distinguishing them.
- Adding a legend helps with multiple curves. Use the option `label=""` with the name of each curve, and **`plt.legend()`** in order to show the legend.

```
numpoints = 100
x = linspace(0,10,numpoints)

ysin = sin(x)
ycos = cos(x)

plt.xlabel("x")
plt.ylabel("y")
plt.plot(x, ysin, "k-", label = 'sin(x)')
plt.plot(x, ycos, "r--", label = 'cos(x)')
plt.legend()
plt.show()
```



Scatter plots

Instead of a $y(x)$ quantity, we can also have two related quantities, where one might want to find some sort of correlation or codependence.

A standard approach is to look at a **scatter plot**, with the two quantities placed along the two axis as points.

For such plot we can use the point option in `plt.plot`, or **`plt.scatter`**, designed specifically for this scope. You can combine them on the same plot as well.

Consider e.g. a file containing a list of temperatures and magnitude (first and second row), file `stars.txt`

```
683.14508541 15.73
683.14508541 17.01
1012.83217289 15.86
1012.83217289 15.98
1012.83217289 16.73
1195.25068152 10.19
1195.25068152 16.56
1289.42232154 17.99
1384.98930374 15.0
1384.98930374 15.38
1384.98930374 15.39
1384.98930374 15.56
1384.98930374 15.64
1384.98930374 16.15
1481.51656803 7.86
```

Scatter plots

```
from numpy import loadtxt
import matplotlib.pyplot as plt

data = loadtxt("stars.txt", float)
x = data[:,0] # The first column from file
y = data[:,1] # The second column from file

plt.scatter(x,y, color = "blue", alpha = 0.25)
plt.xlabel("Temperature")
plt.ylabel("Magnitude")
plt.xlim(0,13000)
plt.ylim(-5,20)
plt.show()
```

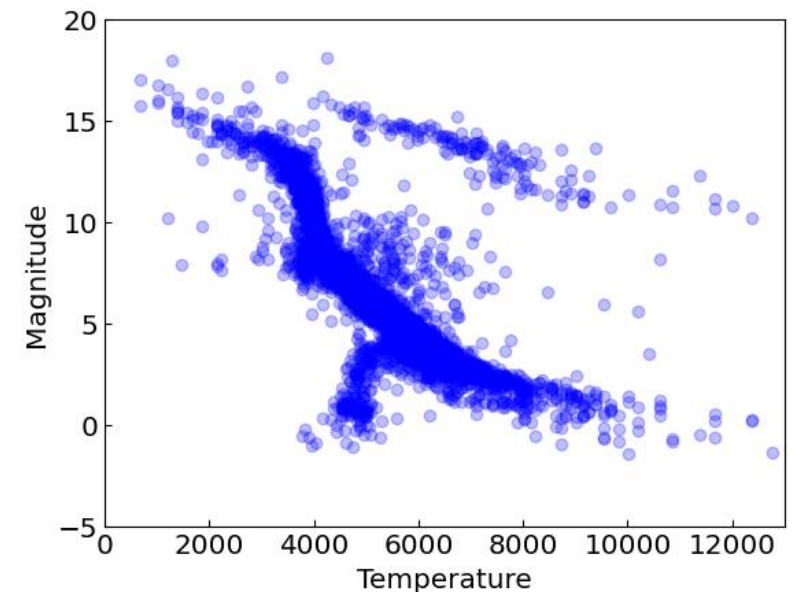
specifying data type

add some transparency.
Helps in seeing regions with higher density

It is called Hertzsprung-Russel diagram

It reveals that most of stars fall along a main band, and red giants and white dwarfs as dots falling off the main sequence.

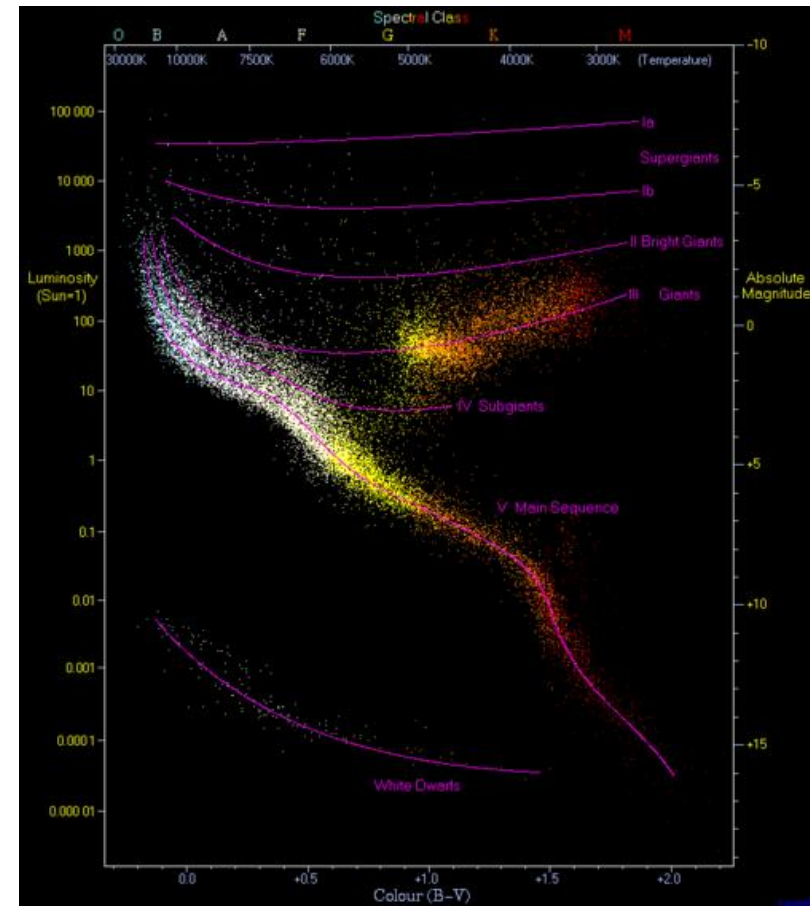
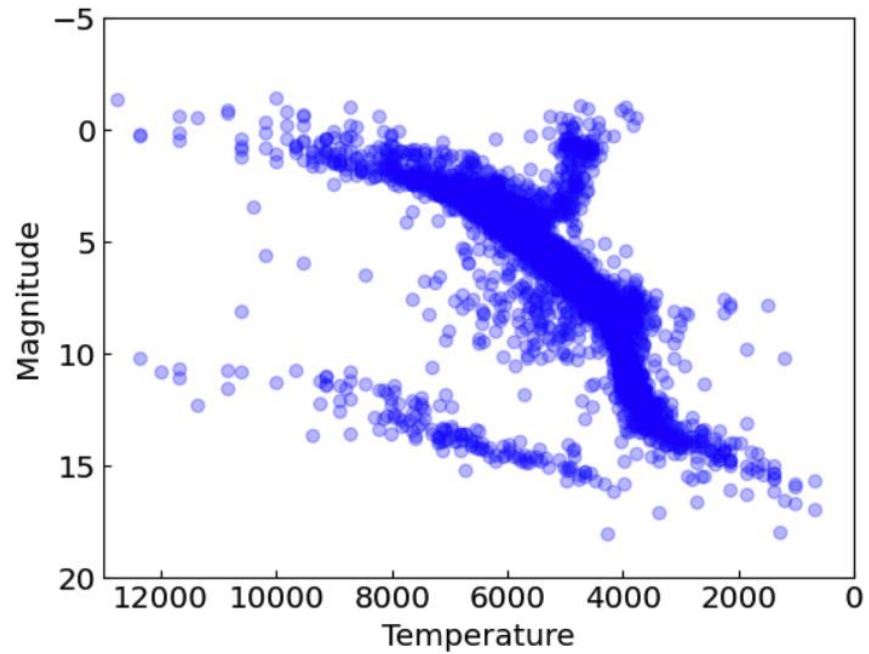
The more luminous (intrinsically bright) an object, the lower its magnitude number (inverse log relation)



Scatter plots

```
plt.xlim(13000,0)  
plt.ylim(20,-5)
```

To have most brightest and hottest stars at the top, we can e.g. revert the axis



Scatter plots

Another example: uniformly distributed points inside a unit square

From matplotlib docs https://matplotlib.org/stable/gallery/shapes_and_collections/scatter.html#sphx-glr-gallery-shapes-and-collections-scatter-py

```
import numpy as np

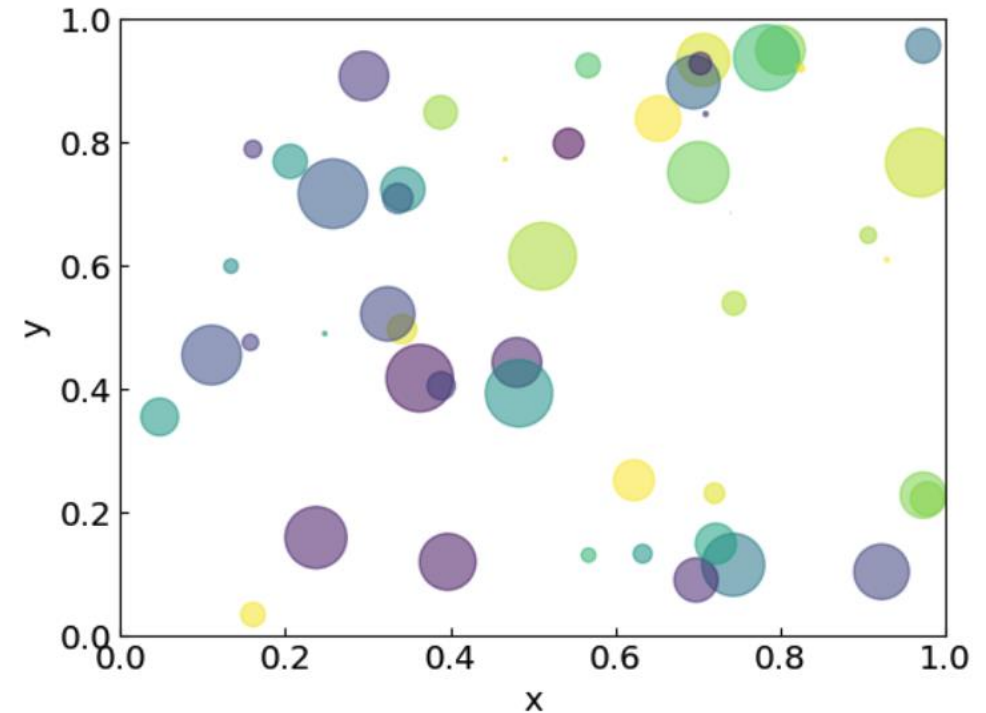
# Fixing random state for reproducibility
np.random.seed(19680801)

N = 50
x = np.random.rand(N)
y = np.random.rand(N)

# Random color
colors = np.random.rand(N)

# Random size
area = (30 * np.random.rand(N))**2 # 0 to 15 point radii

plt.xlim(0,1)
plt.ylim(0,1)
plt.xlabel("x")
plt.ylabel("y")
plt.scatter(x, y, s=area, c=colors, alpha=0.5)
plt.show()
```



Density plots

For situations where we need to plot data dependent on two parameters instead of one, $f(x,y)$, e.g. local charge or local temperature on a surface, distribution of particles incident on a imaging detector, etc.

Density plots or heat maps are two-dimensional plots where color specify the data value. In these plots we have a lateral bar indicating for each color the specified value

Let's start with the file circular.txt containing a simply array of values

```
0.0050 0.0233 0.0515 0.0795 0.1075 ...  
0.0233 0.0516 0.0798 0.1078 0.1358 ...  
0.0515 0.0798 0.1080 0.1360 0.1639 ...  
0.0795 0.1078 0.1360 0.1640 0.1918 ...  
0.1075 0.1358 0.1639 0.1918 0.2195 ...  
...      ...      ...      ...      ...
```

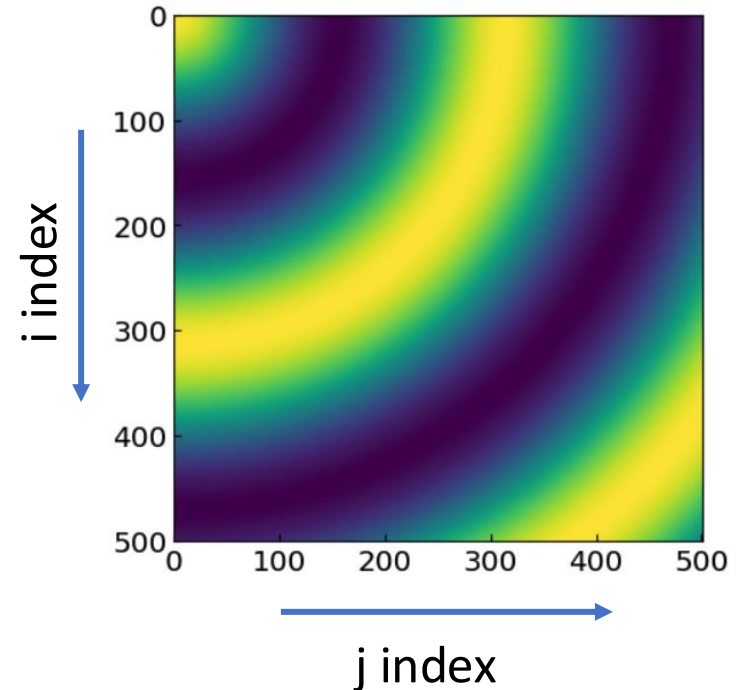
each data represent a $f(x,y)$ value in a grid (where for now we don't specify the range or values of x and y)

Density plots

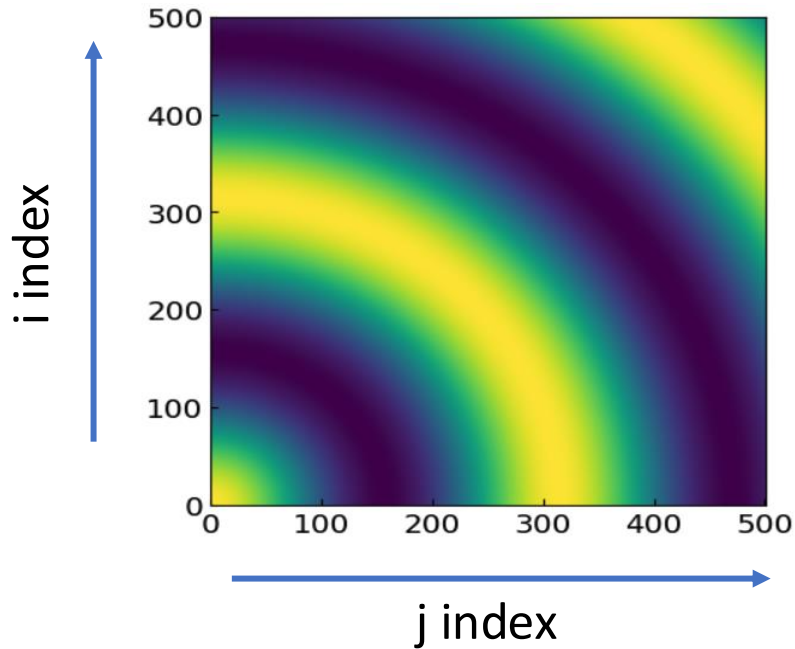
We can produce a nice density plot in the following way with the command **plt.imshow**

```
import matplotlib.pyplot as plt
from numpy import loadtxt
data = loadtxt("circular.txt",float)
plt.imshow(data)
plt.show()
```

The function automatically adjust the color-scale to cover all values in the array. The numbered axis are here just the array indexes i,j for $\text{data}[i,j]$



Density plots

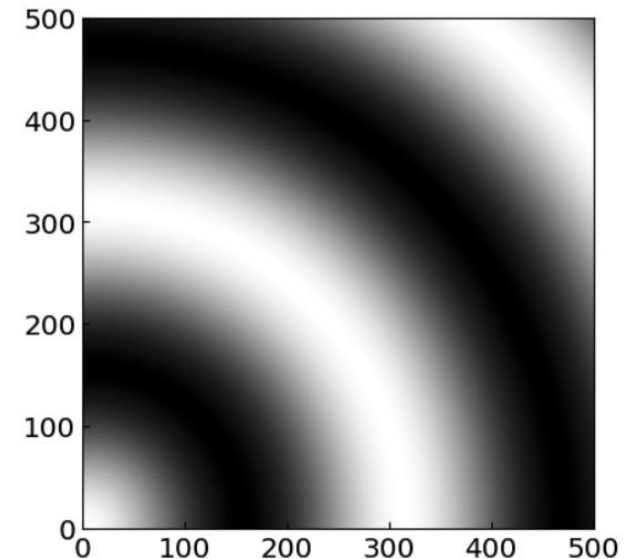


A more natural representation has the y-axis in the opposite direction. To do so, add the following option

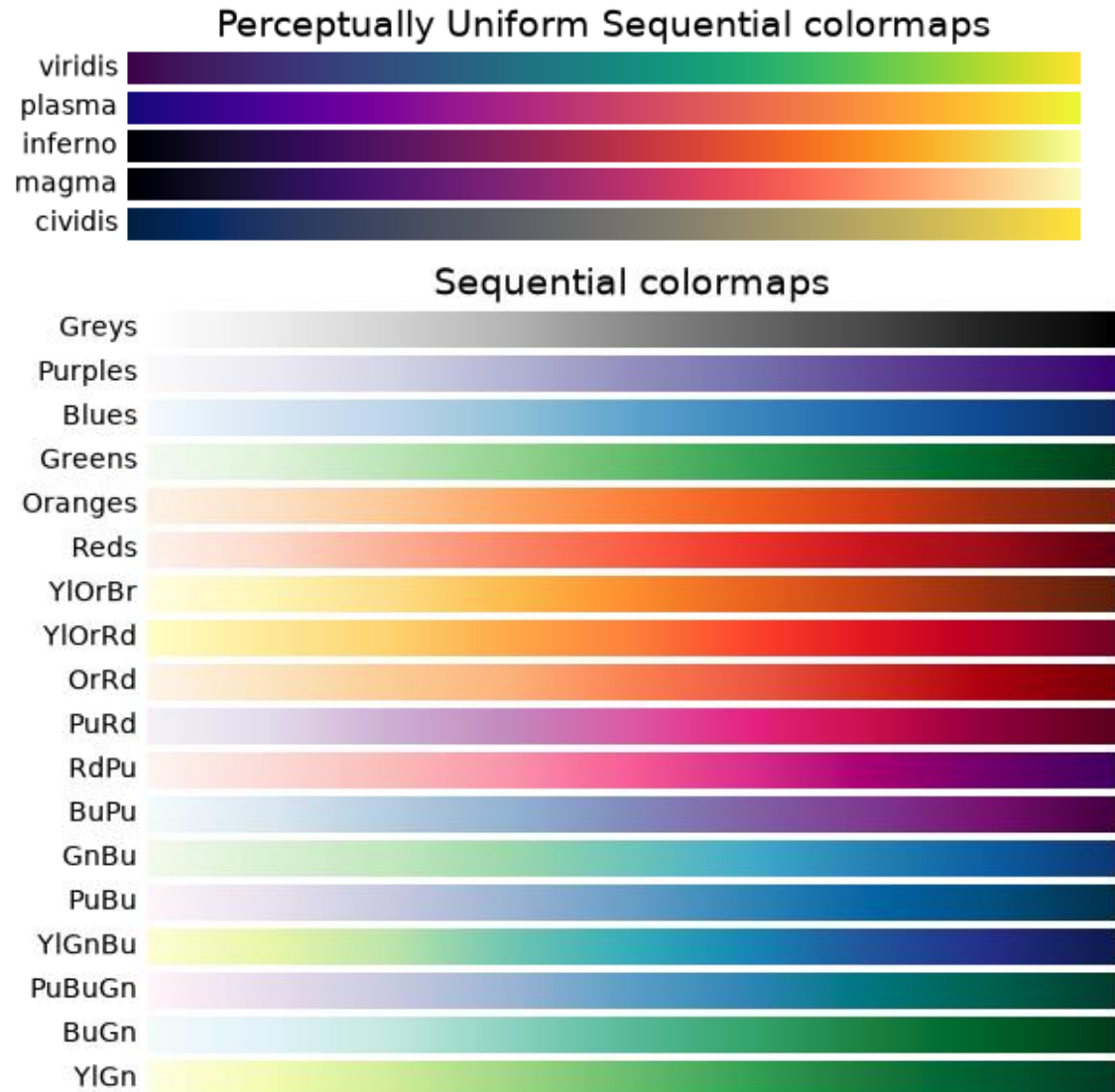
```
plt.imshow(data, origin="lower")
```

Also, you want to keep in mind that usually the first index *i* is what you refer to as the *x* variable, and so when creating the data file you should swap *i* and *j* indexes

The color scheme is the matplotlib default, but you can change it based on the data you want to visualize. Just add before `plt.show` the command `plt.gray()` to change to a grey scale



Density plots



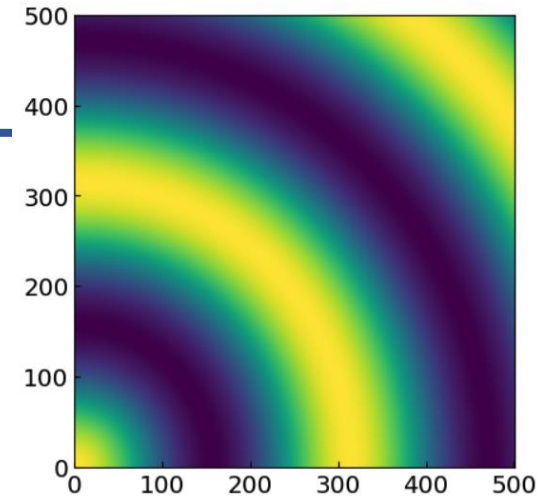
The correct choice of color is important to make the plot readable

<https://matplotlib.org/stable/users/explain/colors/colormaps.html>

Density plots

What else is missing in this plot to make it readable?

- axis labels
- correct range (and not index range)
- value corresponding to colors
- aspect ratio?

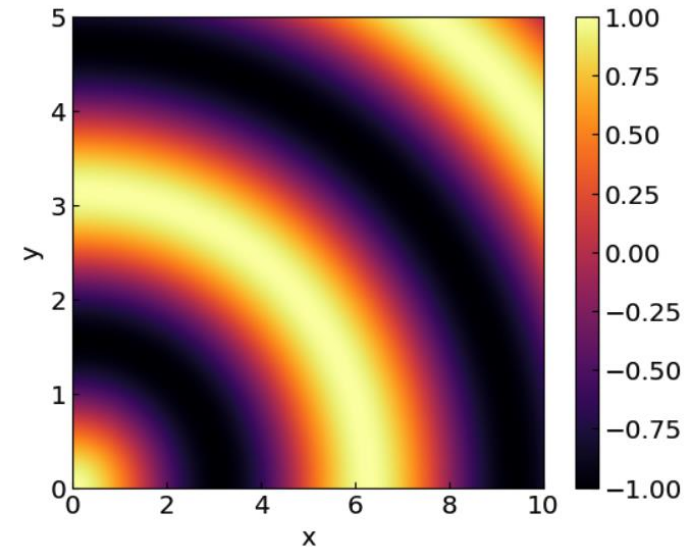


```
import matplotlib.pyplot as plt
from numpy import loadtxt
data = loadtxt("circular.txt", float)
plt.imshow(data, origin="lower", extent=[0, 10, 0, 5], aspect=2)
plt.inferno()
plt.xlabel("x")
plt.ylabel("y")
plt.colorbar()
plt.show()
```

specify correct range of i,j

correct aspect ratio

add lateral colorbar



And so on. You can e.g. limit the range with `plt.xlim` and `plt.ylim`, maybe add the name to the color bar `f(x,y)`, superimpose curves and points with `plt.scatter` and `plt.plot`, etc.

Density plots: example of wave interference

Suppose we drop two pebbles into a pond. At some time, the water ripples of each pebble described by a sine wave, with centers x_1, y_1 and x_2, y_2 and ripple height described by

$$\xi_1(x, y) = \xi_0 \sin kr_1$$

with r_1 and r_2 distances from their centers:

$$\xi_2(x, y) = \xi_0 \sin kr_2$$

$$r_1 = \sqrt{(x - x_1)^2 + (y - y_1)^2}$$

$$r_2 = \sqrt{(x - x_2)^2 + (y - y_2)^2}$$

Take the wavelength of the waves is $\lambda = 5$ cm (with the wavevectors $k = 2\pi/\lambda$), the amplitude $\xi_0 = 1$ cm and the center distances 20 cm.

Compute the total amplitude
and plot it as a density plot

$$\xi(x, y) = \xi_0 \sin kr_1 + \xi_0 \sin kr_2.$$

Different choices to make:

- choice of plot range on x and y
- how to position the center points
- grid size

Density plots: example of wave interference

```
#using a for cycle to initialize our xi matrix
import numpy as np
import matplotlib.pyplot as plt

wavelength = 5.0
k = 2 * np.pi / wavelength
xi0 = 1.0
separation = 20.0
side = 100.0
points = 500
spacing = side / points

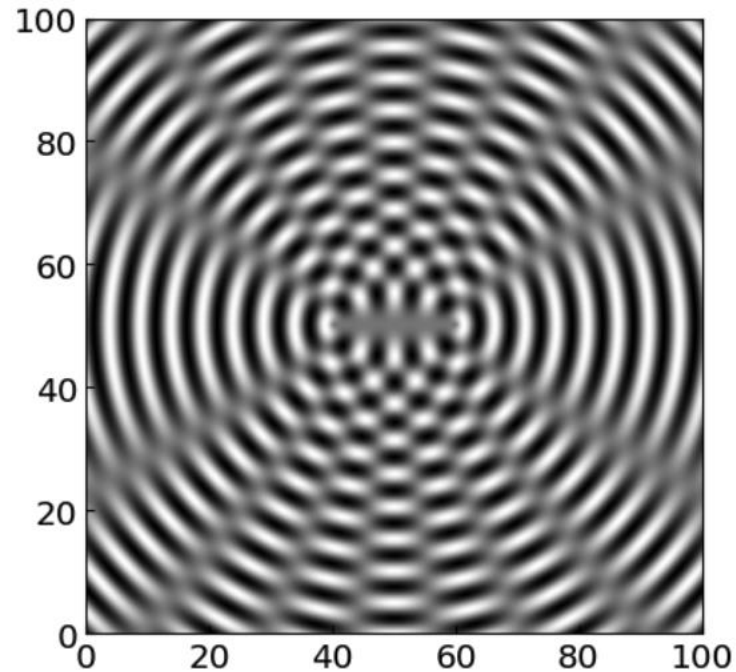
# Position of two sources
x1 = side/2 + separation/2
y1 = side/2
x2 = side/2 - separation/2
y2 = side/2

# Initialize xi array
xi = np.zeros([points, points])

# Compute xi
for i in range(points):
    y = spacing * i
    for j in range(points):
        x = spacing * j
        r1 = np.sqrt((x - x1)**2 + (y - y1)**2)
        r2 = np.sqrt((x - x2)**2 + (y - y2)**2)
        xi[i, j] = xi0 * np.sin(k * r1) + xi0 * np.sin(k * r2)

# Plot
plt.imshow(xi, origin="lower", extent=[0, side, 0, side])
plt.gray()
plt.show()
```

Initialize a 2d array with the values of $\xi(x,y)$ using a for cycle (old fashioned way)



Density plots: example of wave interference

```
import numpy as np
import matplotlib.pyplot as plt

wavelength = 5.0
k = 2 * np.pi / wavelength
xi0 = 1.0
separation = 20.0
side = 100.0
points = 500

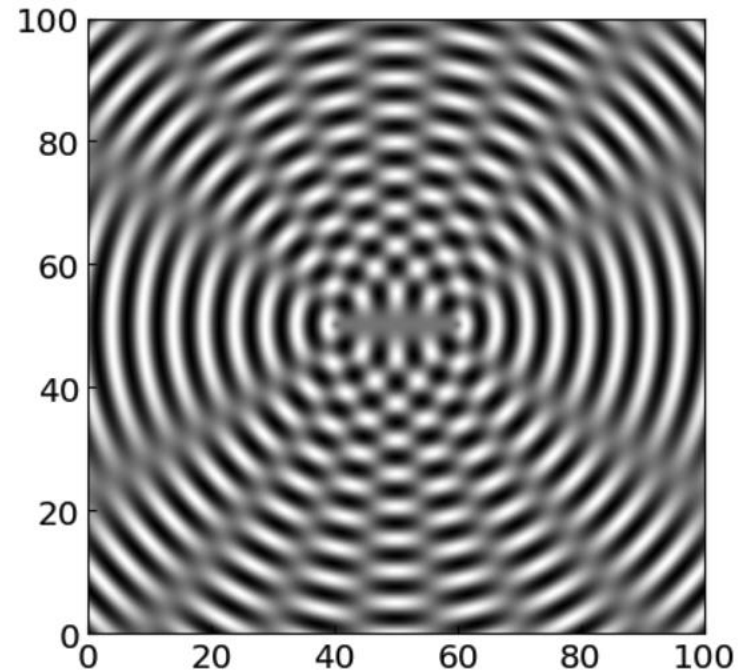
# Grid
x = np.linspace(0, side, points)
y = np.linspace(0, side, points)
#create 2d matrices X,Y each with the values of x and y
X, Y = np.meshgrid(x, y)

# Posizioni delle due sorgenti
x1 = side/2 + separation/2
y1 = side/2
x2 = side/2 - separation/2
y2 = side/2

# Distanze e interferenza
r1 = np.sqrt((X - x1)**2 + (Y - y1)**2)
r2 = np.sqrt((X - x2)**2 + (Y - y2)**2)
xi = xi0 * (np.sin(k * r1) + np.sin(k * r2))

# Plot
plt.imshow(xi, origin="lower", extent=[0, side, 0, side], cmap="gray")
plt.show()
```

a more advanced (equivalent and more efficient version)
doing operations directly on arrays



Contour plots

Another way to display a $f(x,y)$ function is to use contour (or level) lines. Take e.g. an electric dipole potential

$$V_{dip}(x, y) = \frac{1}{\sqrt{(x-x_1)^2+(y-y_1)^2}} - \frac{1}{\sqrt{(x-x_2)^2+(y-y_2)^2}}$$

with (x_1, y_1) and (x_2, y_2) are the coordinates of the poles

```
import matplotlib.pyplot as plt
import numpy as np

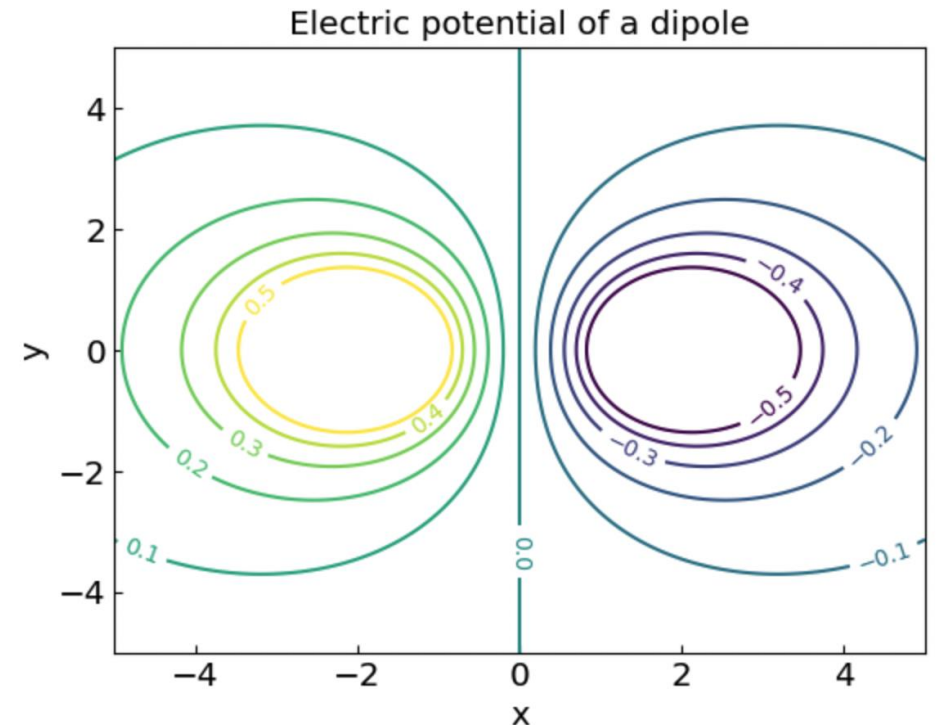
x = np.linspace(-5,5,200)
y = np.linspace(-5,5,200)

x1, y1 = -2., 0.
x2, y2 = 2., 0.

Vdip = 1./np.sqrt((X-x1)**2+(Y-y1)**2) - 1./np.sqrt((X-x2)**2+(Y-y2)**2)

plt.title("Electric potential of a dipole")
plt.xlabel("x")
plt.ylabel("y")

CS = plt.contour(x, y, Vdip, levels = np.linspace(-0.5,0.5,11))
plt.clabel(CS)
plt.show()
```



Histogram

```
import numpy as np
import matplotlib.pyplot as plt

x = np.random.normal(loc=2, scale=0.5, size=10000)
```

normalize histogram (Area=1)

number of bins (good choice is number of elements in x to power 1/3)

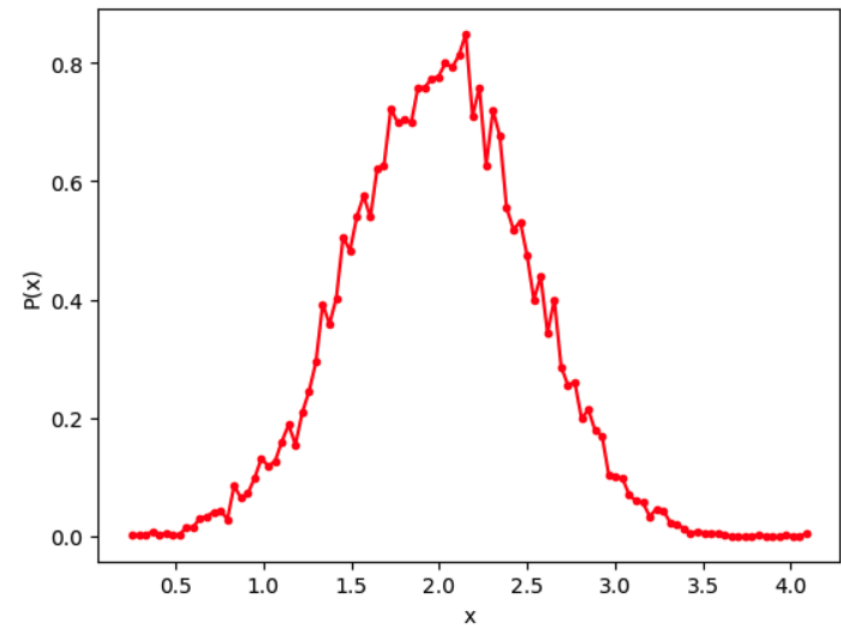
```
hist, bin_edges = np.histogram(x, density=True, bins=100)
```

bin edges

array with histogram value $P(x)$

```
bin_centers = (bin_edges[:-1] + bin_edges[1:]) / 2

plt.plot(bin_centers, hist, 'ro-', ms=3)
plt.xlabel("x")
plt.ylabel("P(x)")
plt.show()
```



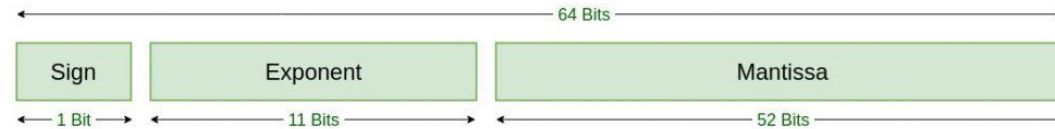
Errors and accuracy

References: *Computational Physics* by Mark Newman, Chapter 4
<https://websites.umich.edu/~mejn/cp2/chapters/chapter4.pdf>
Numerical Recipes Third Edition by W.H. Press et al.

Floating-point number representation

Floating-point, or real, numbers are represented in python by a bit sequence composed in following way, and composing a number in base 2

$$x = S \times M \times 2^E$$



Double Precision
IEEE 754 Floating-Point Standard

- Sign S (+1 or -1)
- Exponent E
- Mantissa M (significant digits)

In this notation (and using base 10) a number is represented in the following way: $-2195.67 = -2.19567 \times 10^3$

This means we have:

- 1) a limit for the smallest (10^{-308}) and largest (10^{308}) representable (through the exponent E)
- 2) a limit to the number of digits representing the mantissa M (~16 decimal digits)

E.g. in the programming language C you have instead the format float (using 1-8-23 digits instead for the representation, thus reducing these two limits and the memory used)

Floating-point number representation

You specify numbers in python using the scientific notation in the following way:

$-2.19567 \times 10^3 = -2.19567e3$ (e is the exponent)

The range in which you can describe your numbers $[-10^{308}, 10^{308}]$ is certainly sufficiently large for our scope.

When coding goes wrong however you can surpass these limits and get “**inf**”. It means the variable has overflowed

For integers instead there is no limit, provided you have enough memory

Numerical error

Let's talk about the limit on the number of digits (16) for the mantissa

Any number, even irrational ones like π or $\sqrt{2}$ which have infinite digits are approximated

True value of π :	3.1415926535897932384626...
Value in Python:	3.141592653589793
Difference:	0.0000000000000002384626...

The difference between the true value of a number and its value on the computer is called the **rounding error** or round-off error

Moreover, in the conversion from and two binary format you get you get your number correctly express up to that precision

$$x=3.3 \quad \text{means} \quad x=3.3+\delta$$

and the error δ can be either positive or negative

Numerical error

Try for example:

```
x = 1.1 + 2.2
```

```
print(x)
```

3.3000000000000003

In fact, if you try an equality test using a float

```
x = 1.1 + 2.2
```

```
print("x = ",x)
```

```
if (x == 3.3):
```

```
    print("x == 3.3 is True")
```

```
else:
```

```
    print("x == 3.3 is False")
```

you will get False

Numerical error

It is thus not a good idea to do equality test with floats. If needed to be done, it is best to check the equality only up to a certain precision

```
# The desired precision
eps = 1.e-12

# The comparison
if (abs(x-3.3) < eps):
    print("x == 3.3 to a precision of",eps,"is True")
else:
    print("x == 3.3 to a precision of",eps,"is False")
```

Numerical error

Typically, one expects an error of the order of the precision in the mantissa 10^{-16} due to round-off errors in the conversion to binary

A good assumption to consider the error to be a (uniformly distributed) random number with standard deviation $\sigma = \epsilon x$, with $\epsilon \simeq 10^{-16}$ with ϵ called machine precision

Thus when e.g. summing up numbers $y = \sum_{i=1}^N x_i$

You expect:

- $\sigma_y \sim \sqrt{N} \epsilon_M$ if errors are independent
- $\sigma_y \sim N \epsilon_M$ if errors are correlated

Numerical error

Although small, there is however a situation where it becomes relevant:

- adding numbers of very different sizes (the smallest numbers typically get lost)
- subtracting numbers which are very close, with the differences comparable with the rounding error

$$\begin{array}{ccc} \begin{array}{l} x = 10000000000000000 \\ y = 100000000000000001.2345678901234 \end{array} & \begin{array}{c} \longrightarrow \\ \text{rounding to} \\ 16 \text{ digits} \end{array} & \begin{array}{l} x = 10000000000000000 \\ y = 10000000000000001.2 \end{array} \longrightarrow \begin{array}{l} y - x = 1.2 \\ \text{instead of} \\ 1.2345678901234 \end{array} \end{array}$$

this phenomenon is called **catastrophic cancellation**

Numerical error: example

Consider e.g. $x = 1$ and $y = 1 + \delta\sqrt{2}$

Symbolically, one has $\delta^{-1}(y - x) = \sqrt{2} = 1.41421356237\dots$

If you test it on a computer for a very small value of $\delta = 10^{-14}$

```
from math import sqrt

delta = 1.e-14
x = 1.
y = 1. + delta * sqrt(2)
res = (1./delta)*(y-x)
print(delta, "* (y-x) = ", res)
print("The accurate value is sqrt(2) = ", sqrt(2))
print("The difference is ", res - sqrt(2))
```

1e-14 * (y-x) = 1.4210854715202004

The accurate value is sqrt(2) = 1.4142135623730951

The difference is 0.006871909147105226

Numerical error: example of quadratic equation

Try to compute the roots of: $ax^2 + bx + c = 0$ using

$$x_{1,2} = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a} \quad \text{and the alternative form} \quad x_{1,2} = \frac{2c}{-b \mp \sqrt{b^2 - 4ac}}$$

and $a = 10^{-4}, b = 10^4, c = 10^{-4}$

You can already notice that $\sqrt{b^2 - 4ac} \sim b$ which brings to the subtraction of similar numbers for one root

Numerical error: example of quadratic equation

Normal form:

```
a = 1.e-4
b = 1.e4
c = 1.e-4

x1 = (-b + sqrt(b*b - 4.*a*c)) / (2.*a)
x2 = (-b - sqrt(b*b - 4.*a*c)) / (2.*a)

print("x1 = ", x1)
print("x2 = ", x2)
```

x1 = -9.094947017729282e-09 instead of -1e-08
x2 = -100000000.0

Alternative form:

```
x1 = 2*c / (-b - sqrt(b*b-4.*a*c))
x2 = 2*c / (-b + sqrt(b*b-4.*a*c))

print("x1 = ", x1)
print("x2 = ", x2)
```

x1 = -1e-08
x2 = -109951162.7776

brings the opposite result, x2 is less precise

Thus, one has to combine the two forms to get both solution accurately

Numerical error: example of derivative

Try now to compute numerically the derivative of $f(x) = x(x - 1)$

using the formal definition $f'(x) \approx \frac{f(x + h) - f(x)}{h}$

Compute the derivative at $x=1$ using small but finite values of h ranging from 1 down to 10^{-16} and compare it to the correct result ($f'(x)=2x-1$ gives $f'(1)=1$)

Plot $f'(x,h)$ for different values of h . (use `plt.xscale('log')` and `plt.yscale('log')` to do a log-log plot)

Numerical error: derivative example

```
import matplotlib.pyplot as plt

def f(x):
    return x*(x-1.)

def df_exact(x):
    return 2.*x - 1.

def df_numeric(x,h):
    return (f(x+h) - f(x)) / h

print("{:<10} {:<20} {:<20}".format('h',"f'(1)","Relative error"))
x0 = 1.

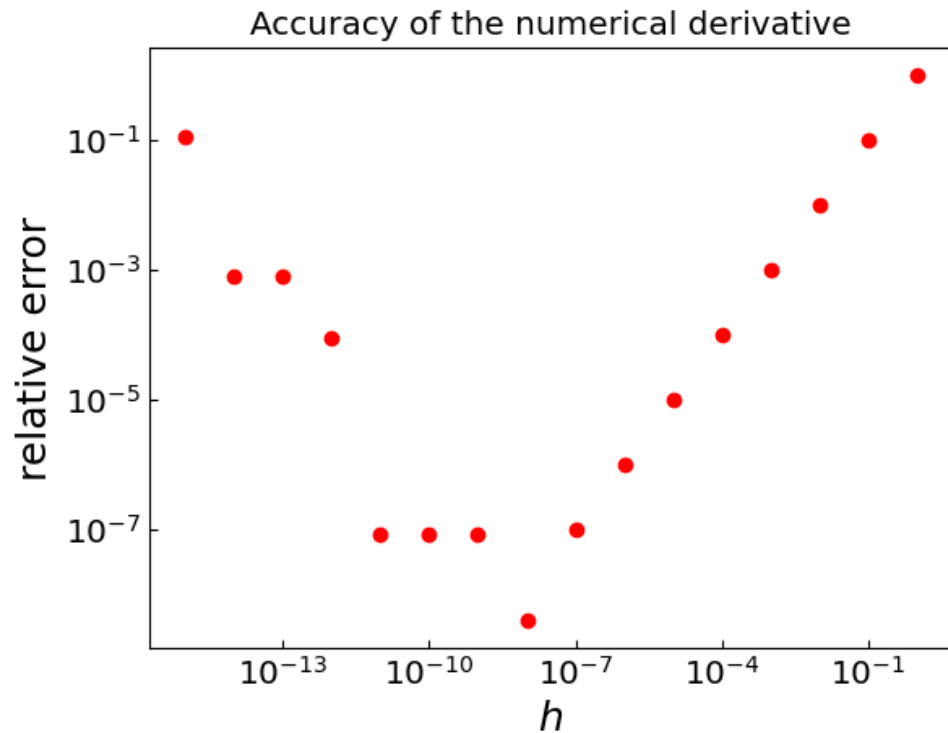
arr_h = []
arr_df = []
arr_err = []

for i in range(0,-20,-1):
    h = 10**i
    df_val = df_numeric(x0,h)
    df_err = abs(df_numeric(x0,h) - df_exact(x0)) / df_exact(x0)
    print("{:<10} {:<20} {:<20}".format(h,df_val,df_err))
    arr_h.append(h)
    arr_df.append(df_val)
    arr_err.append(df_err)

plt.title("Accuracy of the numerical derivative")
plt.xlabel("${h}$", fontsize=18)
plt.ylabel("relative error", fontsize=18)
plt.xscale('log')
plt.yscale('log')
plt.scatter(arr_h, arr_err, color="red")
plt.show()
```

h	f'(1)	Relative error
1	2.0	1.0
0.1	1.1000000000000001	0.100000000000000098
0.01	1.0100000000000001	0.0100000000000000897
0.001	1.00099999999998895	0.00099999999998895337
0.0001	1.00009999999998899	9.99999988985486e-05
1e-05	1.0000100000065513	1.0000006551269536e-05
1e-06	1.0000009999177333	9.99917733279787e-07
1e-07	1.0000001005838672	1.0058386723521551e-07
1e-08	1.0000000039225287	3.922528746258536e-09
1e-09	1.000000083740371	8.374037108183074e-08
1e-10	1.000000082840371	8.284037100736441e-08
1e-11	1.000000082750371	8.275037099991778e-08
1e-12	1.0000889005833413	8.890058334132256e-05
1e-13	0.9992007221627407	0.0007992778372593046
1e-14	0.9992007221626509	0.0007992778373491216
1e-15	1.1102230246251577	0.11022302462515765
1e-16	0.0	1.0
1e-17	0.0	1.0
1e-18	0.0	1.0
1e-19	0.0	1.0

Numerical error: derivative example



The relative error first decreases (as expected) with decreasing h , but then increases again. Why is this?

Round-off error when h becomes too small!

Program speed

References: *Computational Physics* by Mark Newman, Chapter 4
<https://websites.umich.edu/~mejn/cp2/chapters/chapter4.pdf>

Program speed

Computers are not infinitely fast

For typical small problems, using nowadays millions of mathematical operations is not a problem and is a matter of seconds

For billion of operations, we go to minutes. We can still perform them on our computer

What about trillions of operations or more?

We might need to turn to supercomputers, trying to parallelize our operations across multiple processors or for GPUs

Program speed: example of computing Madelung's constant

The Madelung constant gives the total electric potential felt by an atom in a solid.

Consider **solid sodium chloride**—table salt.

Atoms are arranged on a **cubic lattice** with spacing a , with alternating sodium (positive charge $+e$) and chlorine (negative charge $-e$) atoms

Let's label each position in the lattice by three integer coordinates i, j, k , and have the sodium atoms fall at positions where $i + j + k$ is even and the chlorine atoms at positions where $i + j + k$ is odd.

Place a sodium atom at the origin $i = j = k = 0$ and compute the potential in this position.

Given the distance from the origin to the atom at position (i, j, k) $\sqrt{(ia)^2 + (ja)^2 + (ka)^2} = a\sqrt{i^2 + j^2 + k^2}$,

the potential at the origin created by such an atom is $V(i, j, k) = (-1)^{i+j+k} \frac{e/4\pi\epsilon_0}{a\sqrt{i^2 + j^2 + k^2}}$,

and the total potential in 0 is

$$V_{\text{total}} = \sum_{\substack{i, j, k = -L \text{ to } L \\ \text{not } i=j=k=0}} V(i, j, k) = \frac{e}{4\pi\epsilon_0 a} M$$

Madelung constant

Program speed: example of computing Madelung's constant

The Madelung constant M is the dimensionless quantity

$$M = \sum_{\substack{i,j,k=-L \text{ to } L \\ \text{not } i=j=k=0}} \frac{(-1)^{i+j+k}}{\sqrt{i^2 + j^2 + k^2}}.$$

It should be for $L \rightarrow \infty$, but one can get an approximation just by making L large.

But how large? How does it converge? Implement the formula and start from e.g. $L=10$, and increase it

```
from math import sqrt
import time

start = time.perf_counter()
L = 10
M = 0.0
for i in range(-L,L+1):
    for j in range(-L,L+1):
        for k in range(-L,L+1):
            if i==0 and j==0 and k==0:
                continue
            if (i+j+k)%2==0:
                M += 1/sqrt(i*i+j*j+k*k)
            else:
                M -= 1/sqrt(i*i+j*j+k*k)
end = time.perf_counter()
print(f"L = {L} M = {M} Calculation time: {end - start:.4f} sec")
```

Program speed: example of computing Madelung's constant

`L = 10 M = -1.6925789282594415 Calculation time: 0.0043 sec`

`L = 100 M = -1.7418198158396654 Calculation time: 1.2498 sec`

`L = 1000 M = -1.7469875326230075 Calculation time: 1234.6141 sec`

Already at $L=1000$ it takes minutes. In fact, the number of iterations is $(2L+1)^3-1$

For each iteration we are doing 5 operations (squaring numbers, doing the sqrt and doing the reciprocal), so $5[(2L+1)^3-1]$ operations

At $L=100$ we are doing ~40million operations

At $L=1000$ we are scaling to ~40billions.

Take aways:

- $L=1000$ is still doable on your pc, but $L=10000$ takes 1000 times longer, i.e. three weeks
- There is a balance between computational time spent and accuracy. From $L=100$ to $L=1000$ the value does not change, and we might argue it was not necessary
- A simple program as such can easily take forever to finish. Especially when we start nesting loops. always be aware of these loop and how they are scaling. There might be smarter, faster or easier ways to make your estimation that do not take as long.